

1. AIM

To design and implement a Digital Payment System in Java using object-oriented programming concepts, simulating real-world digital payment operations such as wallet creation, money addition, UPI payments, balance checking, and transaction validation.

2. DESCRIPTION

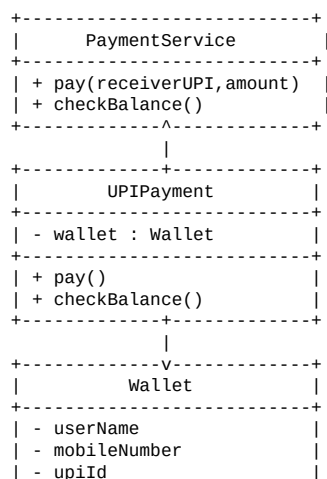
The Digital Payment System is designed for a user with a digital wallet. The system allows the user to create a wallet using a name, mobile number, and UPI ID, add money to the wallet, make payments to another user's UPI ID, check the available balance, and validate transactions before processing them.

The program uses a `PaymentService` interface to define common payment operations. The `Wallet` class stores user information and balance using private data members to demonstrate encapsulation. The `UPIPayment` class implements the `PaymentService` interface and performs UPI payments. Custom exceptions handle invalid UPI IDs, invalid transaction amounts, and insufficient balance.

3. ALGORITHM

1. Start the program.
2. Create `PaymentService` and declare `pay()` and `checkBalance()`.
3. Create `Wallet` with private `userName`, `mobileNumber`, `upiId`, and `balance`.
4. Create a constructor and initialize wallet details and balance.
5. Define `addMoney()`, `getBalance()`, and `displayWalletDetails()`.
6. Create custom exceptions for invalid UPI, invalid amount, and insufficient balance.
7. Create `UPIPayment` and implement `PaymentService`.
8. Associate a `Wallet` object with `UPIPayment`.
9. Validate the receiver UPI ID using String operations.
10. Throw `InvalidUPIException` if the UPI ID is invalid.
11. Throw `InvalidAmountException` if the amount is zero or negative.
12. Throw `InsufficientBalanceException` if the balance is insufficient.
13. Deduct the amount when all validations succeed.
14. Display the payment success message.
15. Use try-catch-finally for exception handling.
16. Display the final wallet details and balance.
17. Stop the program.

4. UML DIAGRAM



```

| - balance |
+-----+
| + addMoney() |
| + getBalance() |
| + deductMoney() |
| + displayWalletDetails() |
+-----+

```

5. SOURCE CODE

```

package core_java;

interface PaymentService {
    void pay(String receiverUPI, double amount)
        throws InvalidUPIException, InvalidAmountException,
            InsufficientBalanceException;
    void checkBalance();
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) { super(message); }
}

class InvalidUPIException extends Exception {
    InvalidUPIException(String message) { super(message); }
}

class InvalidAmountException extends Exception {
    InvalidAmountException(String message) { super(message); }
}

class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    Wallet(String userName, String mobileNumber, String upiId) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = 0.0;
    }

    void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0)
            throw new InvalidAmountException("Amount must be greater than zero.");
        balance = balance + amount;
        System.out.println("Money added successfully: Rs." + amount);
    }

    double getBalance() { return balance; }
    void deductMoney(double amount) { balance = balance - amount; }

    void displayWalletDetails() {
        System.out.println("\n----- Final Wallet Details -----");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Balance      : Rs." + balance);
    }
}

class UPIPayment implements PaymentService {
    private Wallet wallet;

    UPIPayment(Wallet wallet) { this.wallet = wallet; }

    public void pay(String receiverUPI, double amount)
        throws InvalidUPIException, InvalidAmountException,
            InsufficientBalanceException {

        if (receiverUPI == null || !receiverUPI.contains("@") ||
            receiverUPI.startsWith("@") || receiverUPI.endsWith("@"))
            throw new InvalidUPIException("Invalid UPI ID: " + receiverUPI);

        if (amount <= 0)
            throw new InvalidAmountException(
                "Payment amount must be greater than zero.");

        if (amount > wallet.getBalance())
            throw new InsufficientBalanceException(
                "Insufficient balance. Available balance: Rs."
                + wallet.getBalance());

        wallet.deductMoney(amount);

        System.out.println("\n----- Payment Details -----");
        System.out.println("Payment Successful!");
        System.out.println("Receiver UPI : " + receiverUPI);
        System.out.println("Amount Paid  : Rs." + amount);
    }
}

```

```

        public void checkBalance() {
            System.out.println("Available Balance: Rs." + wallet.getBalance());
        }
    }

    public class DigitalPaymentSystem {
        public static void main(String[] args) {
            Wallet wallet = new Wallet("Ananya", "9876543210", "ananya@upi");
            PaymentService payment = new UPIPayment(wallet);

            try {
                wallet.addMoney(5000);
                payment.checkBalance();
                payment.pay("kiran@upi", 1500);
            }
            catch (InvalidUPIException e) {
                System.out.println("Transaction Failed: " + e.getMessage());
            }
            catch (InvalidAmountException e) {
                System.out.println("Transaction Failed: " + e.getMessage());
            }
            catch (InsufficientBalanceException e) {
                System.out.println("Transaction Failed: " + e.getMessage());
            }
            finally {
                System.out.println("\nTransaction processing completed.");
            }

            wallet.displayWalletDetails();
        }
    }
}

```

6. OUTPUT SCREENSHOT

```

C:\Java> javac DigitalPaymentSystem.java
C:\Java> java core_java.DigitalPaymentSystem

Money added successfully: Rs.5000.0
Available Balance: Rs.5000.0

----- Payment Details -----
Payment Successful!
Receiver UPI : kiran@upi
Amount Paid : Rs.1500.0

Transaction processing completed.

----- Final Wallet Details -----
User Name      : Ananya
Mobile Number  : 9876543210
UPI ID        : ananya@upi

```

The above terminal-style output shows the result after compiling and executing the Java program.

7. RESULT

Thus, the Digital Payment System was successfully designed and implemented in Java. The program demonstrates interface implementation, encapsulation, constructors, object creation, String-based UPI validation, custom exceptions, try-catch-finally exception handling, wallet operations, UPI payment processing, balance checking, and transaction validation. The final wallet balance after adding Rs.5000 and paying Rs.1500 is Rs.3500.